

Teradata

In-Database Graph Functions

Field Developed Solution

Author(s):

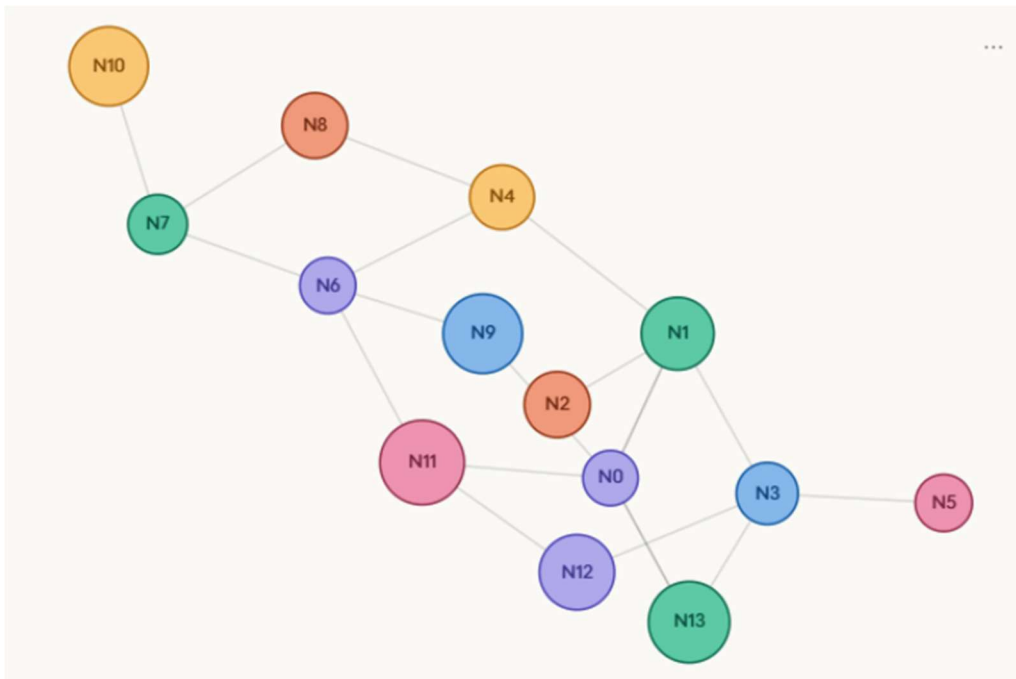
David Chan

Distinguished Data Scientist, Global Data Science, Teradata

Karthik Guruswamy

Financial Services AI Strategy Lead, Teradata

Version: 0.0.6



Executive Summary

EXECUTIVE SUMMARY

INTRODUCTION

USE CASES

ARCHITECTURE

DATA

IMPLEMENTATION

HOW TO USE

CONCLUSION

Overview

As organizations manage increasingly interconnected data — from customer relationships and supply chains to fraud networks and knowledge graphs — the ability to analyze complex relationships at scale has become a strategic priority. Traditional relational databases were not designed for this class of problem. In-database graph functionality addresses this gap by embedding native graph processing directly within the enterprise data platform, eliminating the need to extract, transform, and move data to a separate graph engine.

The Problem

Relationship-intensive analytics today require organizations to maintain a separate graph database alongside their primary data warehouse or analytical platform. This introduces significant friction: data pipelines must be built and maintained, data is duplicated across systems, governance and security policies must be enforced in multiple places, and results from graph queries cannot be easily joined back to transactional or dimensional data. The result is higher infrastructure cost, increased latency, and added complexity for data engineering teams.

The Solution

In-database graph functions bring graph traversal, path analysis, and network analytics natively into the SQL execution environment. Users can express graph queries — such as shortest path, community detection, centrality scoring, and subgraph matching — using familiar SQL syntax or purpose-built graph extensions, operating directly on data where it already lives. No data movement. No separate system to manage. No extra cost to spend.

Introduction

Graph analytics unlock a category of insight that traditional relational queries cannot easily express: how entities are connected, how influence or risk propagates through a network, and which nodes hold structural importance across thousands or millions of relationships. Yet for most enterprise data teams, graph analysis has meant moving data out of the warehouse and into a separate, specialized system — adding pipeline complexity, duplicating data, and fragmenting governance.

This solution eliminates that tradeoff.

Teradata Stored Procedures

Each graph algorithm is packaged as a stored procedure. The stored procedure accepts input parameters — such as the source table, node and edge column names, a starting node, or a depth limit — and encapsulates all execution logic within Teradata. This means graph functions are callable like any other database procedure, can be granted permissions through standard Teradata role management, and are versioned and deployed as database objects.

Python Wrapper

TeradataGE (Teradata Graph Engine) is a Python package that wraps the stored procedure calls and exposes each graph algorithm as a native Python method. It is designed as a companion to the `teradataml` package — the official Teradata Python library for in-database analytics — and follows the same connection model, DataFrame conventions, and API style that `teradataml` users are already familiar with.

TeradataGE accepts a `teradataml` context or an active `TeradataContext` as its entry point, meaning no separate connection setup is required. Results are returned as `TeradataDataFrame` objects, making them immediately composable with `teradataml`'s feature engineering, model scoring, and data preparation functions.

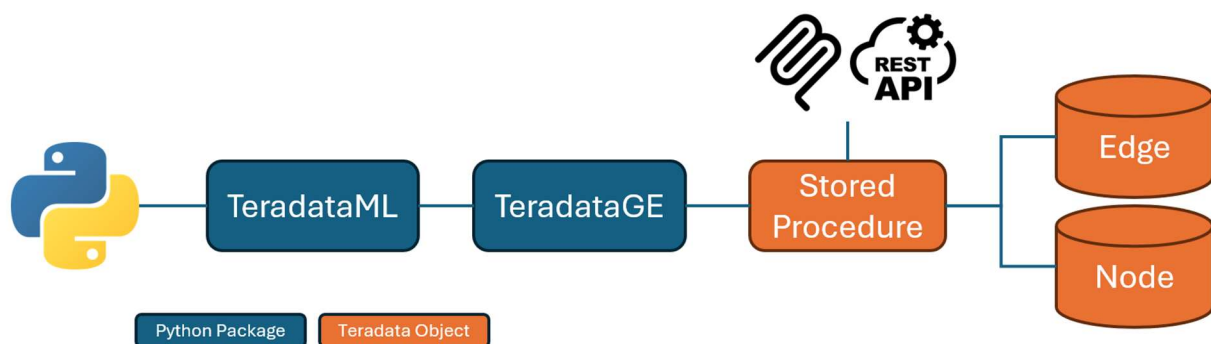


Figure 1: How to use in-database graph functions

Use Cases

Graph analytics is not a single-purpose technology — it is a versatile framework that surfaces value across industries and disciplines. The following use cases illustrate how organizations are applying graph analytics to solve problems that traditional relational or tabular approaches struggle to address.

Anti-Money Laundering (AML)

- Regulatory compliance requires financial institutions to understand not just who their customers are, but who their customers are connected to. Graph analytics enables compliance teams to trace the full ownership structure of corporate entities — including shell companies and beneficial ownership chains — that may span dozens of jurisdictions.
- In AML investigations, graph traversal can follow the flow of funds across multiple hops, revealing layering schemes that would be invisible in a flat transaction log. What might take an investigator week to untangle manually can be surfaced in seconds through graph-based path analysis.

Key graph techniques: Graph traversal, hierarchical relationship modeling, centrality analysis.

Network Operations and Optimization

- Telecom networks are, at their core, graphs — nodes representing equipment and edges representing connections. Graph analytics enables network engineers to model coverage, simulate failure scenarios, and optimize routing in ways that static topology maps cannot.
- When an outage occurs, graph-based root cause analysis can pinpoint the origin and calculate the scope of impact across dependent services in seconds. Capacity planning teams use graph algorithms to identify bottlenecks and model the effect of infrastructure investments before deployment.

Key graph techniques: Shortest path, maximum flow, minimum spanning tree, impact propagation.

Knowledge Graphs and Enterprise Search

- Enterprises accumulate vast amounts of unstructured knowledge — in documents, wikis, emails, and databases — that is difficult to discover and connect. Knowledge graphs organize this information into a structured web of entities and relationships, enabling semantic search and intelligent question answering.
- Organizations are deploying knowledge graphs to power internal search engines that return not just documents, but contextually relevant answers. Combined with large language models (LLMs), graph-augmented retrieval (often called GraphRAG) significantly improves the accuracy and explainability of AI-generated responses in enterprise settings.

Key graph techniques: Knowledge graph construction, entity linking, graph-augmented retrieval (GraphRAG), ontology modeling.

Social Network Analysis and Influence Mapping

- Whether in marketing, public policy, or investigative research, understanding how information and influence flow through a network is enormously valuable. Graph analytics enables organizations to identify key influencers, detect coordinated inauthentic behavior, and model how ideas or products will spread through a population.
- Marketing teams use influence graph analysis to design more effective seeding strategies for product launches. Public health agencies have applied the same techniques to model disease transmission and optimize intervention strategies.

Key graph techniques: Centrality algorithms (Betweenness, PageRank), Community Detection, and Diffusion Modeling.

Data Lineage and Pipeline Dependency Analysis

- In modern data platforms, raw data flows through dozens of transformation steps — ingestion jobs, dbt models, Spark pipelines, BI dashboards — before becoming a business metric. When a source table changes schema or a pipeline fails, understanding which downstream reports and models are affected is critical.
- Graph analytics models data assets (tables, views, models, dashboards) as nodes and data flow as directed edges. Data engineers can instantly trace lineage in both directions: upstream to find the root cause of a data quality issue, and downstream to assess the impact of a schema change before it is deployed. This is foundational to reliable DataOps and effective data governance.

Key graph techniques: Directed Graph Traversal, Transitive Closure, Centrality Analysis, Node Classification.

Architecture

Overview

The in-database graph function architecture is built entirely within the Teradata platform. No external graph engine, no data movement, and no additional infrastructure are required. Graph algorithms execute as SQL workloads inside the Teradata engine, leveraging the platform's MPP architecture for parallelism, volatile temporary tables for session-scoped intermediate state, and stored procedures as the callable unit of execution.

The architecture has four layers that work in concert: the **interface layer** (SQL or TeradataGE Python), the **procedure layer** (stored procedures), the **execution layer** (recursive CTEs running across AMPs), and the **storage layer** (source tables and volatile temp tables).

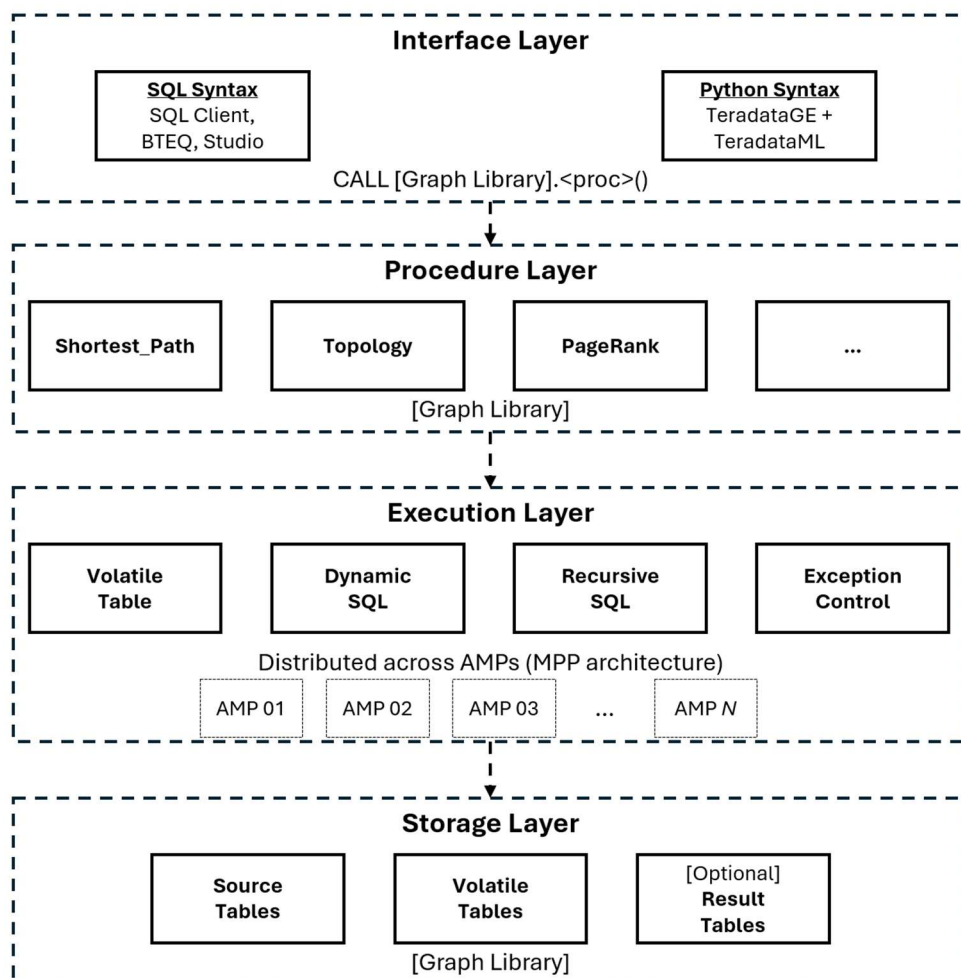


Figure 2: In-database Graph Engine Architecture

Layer 1 — Interface Layer

Users interact with the graph library through two equivalent interfaces. Both invoke the same stored procedures and produce identical results.

- **SQL interface** — any SQL client connected to Teradata (Teradata Studio, BTEQ, DBeaver, Jupyter with Teradata SQL kernel) can call graph functions using a standard CALL statement. This makes graph functions accessible to SQL analysts without any Python dependency.
- **TeradataGE Python interface** — the TeradataGE package wraps stored procedure calls as Python methods and integrates with an active teradataml context. Results are returned as TeradataDataFrame objects. This interface is suited for data scientists working in notebooks or automated ML pipelines.
- Both interfaces pass identical parameters to the procedure layer and receive the same output structures.

Layer 2 — Procedure Layer

Graph algorithms are deployed as Teradata Stored Procedures in a dedicated database (graph_db). Each stored procedure is responsible for:

- **Validating input parameters** — table names, column names, node identifiers, and algorithm-specific settings such as iteration limits or convergence thresholds
- **Initializing volatile temporary tables** — creating the session-scoped working tables the algorithm requires before execution begins
- **Orchestrating recursive execution** — invoking the recursive CTE logic and managing iteration state across steps
- **Writing final results** — materializing the output into a volatile result table or returning a result set to the caller
- **Cleaning up intermediate state** — dropping working volatile tables at procedure completion to release spool space

Stored procedures are granted execute permissions through standard Teradata role-based access control. A user or role with EXECUTE PROCEDURE privilege on graph library database can call any graph function. No direct table access to intermediate working tables is required or exposed.

Layer 3 — Execution Layer

Each stored procedure is constructed with the following components:

Volatile tables

- Each stored procedure creates volatile temporary tables scoped to the current database session. Volatile tables exist only for the duration of the session and are automatically dropped when the session closes. They are written to Teradata spool space and are private to the session — two users running the same graph procedure simultaneously will each have their own isolated set of working tables with no conflict.

Dynamic queries

- Each SP is building and executing a SQL string at runtime rather than writing it as a fixed statement at compile time.
- It supports dynamic input tables structure.

Recursive queries

- Graph traversal is implemented using recursive query. A recursive CTE consists of two parts joined by UNION ALL:
 - Anchor query — defines the starting state of the algorithm (e.g., the seed node at iteration 0, or all nodes initialized with a starting score)
 - Recursive term — defines how the state expands at each subsequent step (e.g., join the current frontier to the edge table to discover next-hop neighbors, or propagate scores to adjacent nodes)

Exception Control

- Exception control in Teradata stored procedures is handled through a structured condition/handler mechanism, similar to try-catch in application languages.
- Some exceptions can be ignored. For example, dropping a table if not exists
- Other critical exceptions will stop the process and return the corresponding SQL error message.

MPP Parallelism

Teradata distributes table rows across AMPs according to each table's Primary Index (PI). In-database graph functions are designed to align edge table distribution with the traversal access pattern:

- Edge tables are recommended to be PI-distributed on the source node column. This ensures that all outgoing edges for a given node reside on the same AMP, allowing local join during each traversal step and minimizing inter-AMP redistribution.
- At each recursive iteration, the engine expands the frontier in parallel across all AMPs simultaneously. Each AMP processes the subset of nodes and edges it holds, and partial results are merged across AMPs before the next iteration begins.
- For algorithms requiring global aggregation — such as PageRank score normalization or convergence checking — Teradata's built-in parallel aggregation handles cross-AMP reduction without requiring application-level coordination.

This design means that graph traversal scales horizontally with the Teradata system. Larger graphs are handled by distributing more data across more AMPs, not by adding external graph infrastructure.

Layer 4 — Storage Layer

Source Tables

Graph functions operate on standard Teradata permanent tables. The typical inputs are:

Table	Contents	Optional	Recommended PI
Edge	One row per directed relationship (source node, target node)	N	Source Node ID
	Weight / Probability	Y	
	Edge Type	Y	
	Edge Attribute(s)	Y	
Node	One row per node	Y	Node ID
	Node Name	Y	
	Node Type	Y	
	Node Attribute(s)	Y	

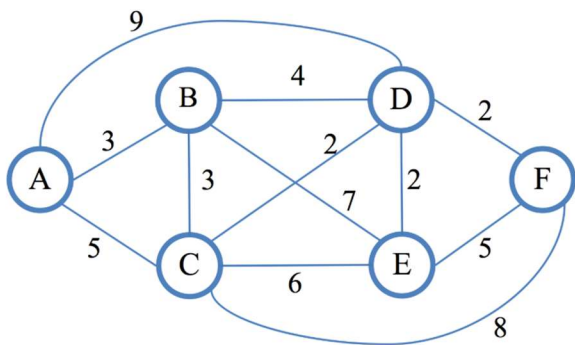
Result Table

- Node ID have to be either INTEGER or BIGINT type
- Optional Result Table can be created at the end of Stored Procedure process. If the SP call does not include output table, the result set will be returned directly from SP.
- Result table can be either perm or volatile table.

Data

In-database graph analysis requires at least one table or view as edge input. For visualization purposes, it is highly recommended to also have a node table or view.

Data Structure:



Integer	VARCHAR		
NodeID	NodeName	Attr1	Attr2
1	A		
2	B		
3	C		
4	D		
5	E		
6	F		

Node Table

Integer	Integer	Float			
FromID	ToID	Weight	Attr1	Attr2	Attr3
1	2	3			
1	3	5			
1	4	9			
2	3	3			
2	4	4			
2	5	7			
3	4	2			
3	5	6			
4	5	2			
4	6	2			
5	6	5			

Edge Table

Implementation

Prerequisites:

- **Database:**
 - Teradata database V17.20 or above.
 - A database *[GraphDB]* for Stored Procedure Installation.
 - Database Privileges for installer.
 1. GRANT CREATE Procedure on *[GraphDB]* to *[Installer]*;
 2. GRANT EXECUTE Procedure on *[GraphDB]* to *[GraphDB]*;
 3. GRANT ALL on *[TargetDB]* to *[GraphDB]* with grant option;
 - Database Privileges for user.
 1. GRANT EXECUTE Procedure on *[GraphDB]* to *[User]*;
- **Client:**
 - Python 3.8 or above
 - The following packages are required:
 1. teradataml>=20.0.0.9
 2. teradatasql>=20.0.0.40
 3. pandas>=2.2.0
 4. pathlib>=1.0.1
 5. plotly>=6.3.1

Installation:

- Download and install TeradataGE python package from PyPI:
 - Command: **pip install teradatage**
- Install Graph Analysis SPs from python with TeradataML and TeradataGE packages:

```
# Teradata python package
from teradataml import *
import getpass
from TeradataGE import td_graph_function, configure, install_db_objects

# Database locator: where graph functions are being installed
configure.graph_install_location = "GraphLib"

# Connect to Teradata database
hostname = "xx.xx.xx.xx"
user = input(prompt="Username for SPs instalation:")
password = getpass.getpass(prompt="Database password for {user}:")
logmech = "TD2"
eng = create_context(host = hostname, username = user, password = password, logmech = logmech)
print(eng)

# Install stored procedures
install_db_objects.install_graph_functions()

# Disconnect from database
remove_context()
```

How to use

Class: Graph Object

TeradataGE's "**td_graph_object**" defines the schema for graph object. Graph object must be defined by calling the corresponding functions. Here are parameters:

Parameter	Description	Default
database_name	Databasename for source and target tables	Current database
edge_table_name	Table/view name for EDGE	
edge_from_node_column_name	Column name for the source node id in edge table	
edge_to_node_column_name	Column name for the target node id in edge table	
edge_type_column_name	Column name for Edge Type	None
edge_weight_column_name	Column name for weight or probability	None
datetime_column_name	Date or Time stamp column for temporal process	None
weight_type	If weight column is defined, weight type should be W (for weight) or P (for probability).	W
edge_attributes	List of attributes column(s) for edge	None
node_table_name	Table/view name for NODE	None
node_id_column_name	node id column name	None
node_type_column_name	node type	None
node_label_column_name	Node name label (for display)	None
source_id	List of starting node id (can be added later)	None
target_id	List of targeting node id (can be added later)	None

Example 1: Weighted Network with Edge only

```
graph_obj = td_graph_function.td_graph_object(edge_table_name = 'graph_edge',
                                              edge_from_node_column_name = 'from_node',
                                              edge_to_node_column_name = 'to_node',
                                              edge_weight_column_name = 'edge_weight'
                                              )
```

Example 2: Unweighted Network with Node and Edge

```
graph_obj = td_graph_function.td_graph_object(edge_table_name = 'imdb_graph_edge',
                                              edge_from_node_column_name = 'from_id',
                                              edge_to_node_column_name = 'to_id',
                                              node_table_name = 'imdb_graph_node',
                                              node_id_column_name = 'node_id',
                                              node_label_column_name = 'primaryname'
                                              )
```

Function: Node and Edge information

Function: edge_info & node_info

Description: Once the graph object is defined, both edge and node information can be retrieved.

Parameter: None

Example:

Python	# Display the defined information for edge print(graph_obj.edge_info())
Return	Edge Table [imdb_graph_edge]: From[from_id] To[to_id] with No Weight. # of edges = 99600 & # of nodes used = 17989.
Python	# Display the defined information for node print(graph_obj.node_info())
Return	Node Table [imdb_graph_node]: Node ID [node_id]. # of Nodes = 17989.

Function: Set Source IDs

Function: set_node_id_from_label_source

Description: Find and set the source ID(s) from node description

Parameter	Description	Default
label	Source ID search based on node's label. '%' can be used for matching sequence. "ALL" represents all the node possible IDs.	
Nodetype	Search based on specific node type.	None
If_exists	replace: replace existing source IDs. append: added into previous existing source IDs.	replace
show_query	Show the actual query	False

Example:

Python	graph_obj.set_node_id_from_label_source("The Last of the Mafia")
Python	graph_obj.set_node_id_from_label_source("%Mafia%")

Function: Set Target ID

Function: set_node_id_from_label_target

Description: Find and set the target ID from node description

Parameter	Description	Default
label	Target ID search based on node's label. '%' can be used for matching sequence.	
show_query	Show the actual query	False

Example:

Python	graph_obj.set_node_id_from_label_target("Midnight Madness")
---------------	---

Function: Topological Order (Topology)

Function: topology

Description: Find and set the target ID from node description

Parameter	Description	Default
source_id	Starting Node IDs. If not defined, use the existing source IDs	None
edge_pattern	Specific each pattern. If none, use all possible edges	None
max_path_length	It is a traversal depth-limiting parameter used in graph engines to control how many levels (hops) deep a graph traversal algorithm will explore from a starting node.	100
weight_filter	Optional search condition based on weight column. Example: '<0.01'	None
return_data	What kind of data is returned at the end. P : network paths N : Impacted nodes	P
output_table	Name of output table. If none, result will be returned from SP directly	None
temp_output_table	True: Output as volatile temporary table False: Output as perm table	True
show_query	Show the actual query	False

Example:

Python	<pre>graph_obj = td_graph_function.td_graph_object(database_name = 'graphdb', edge_table_name = 'graph_edge', edge_from_node_column_name = 'from_node', edge_to_node_column_name = 'to_node', edge_weight_column_name = 'edge_weight', node_table_name = 'graph_node', node_id_column_name = 'id', node_label_column_name = 'node_name') # Directed Path all_path_from_1node = graph_obj.td_topology(source_id = 1, return_data = 'P', output_table = 'example_topology_path_vt') all_path_from_1node</pre>																																	
Return	<table><tr><th>fullpath</th><th>weight</th><th>path_level</th></tr><tr><td>1,3,5</td><td>5.8</td><td>2</td></tr><tr><td>1,3,5,7</td><td>12.8</td><td>3</td></tr><tr><td>1,2,5,7</td><td>12.8</td><td>3</td></tr><tr><td>1,3,6,7</td><td>9.0</td><td>3</td></tr><tr><td>1,2,4,7</td><td>16.5</td><td>3</td></tr><tr><td>1,2,4</td><td>6.5</td><td>2</td></tr><tr><td>1,3,6</td><td>6.5</td><td>2</td></tr><tr><td>1,2</td><td>3.0</td><td>1</td></tr><tr><td>1,3</td><td>3.0</td><td>1</td></tr><tr><td>1,2,5</td><td>5.8</td><td>2</td></tr></table>	fullpath	weight	path_level	1,3,5	5.8	2	1,3,5,7	12.8	3	1,2,5,7	12.8	3	1,3,6,7	9.0	3	1,2,4,7	16.5	3	1,2,4	6.5	2	1,3,6	6.5	2	1,2	3.0	1	1,3	3.0	1	1,2,5	5.8	2
fullpath	weight	path_level																																
1,3,5	5.8	2																																
1,3,5,7	12.8	3																																
1,2,5,7	12.8	3																																
1,3,6,7	9.0	3																																
1,2,4,7	16.5	3																																
1,2,4	6.5	2																																
1,3,6	6.5	2																																
1,2	3.0	1																																
1,3	3.0	1																																
1,2,5	5.8	2																																

Function: Shortest Path

Function: `td_shortest_path`

Description: Find the shortest path from source node to target node with or without weight

Parameter	Description	Default
source_id	Source Node ID. If not defined, use the existing source ID	None
Target_id	Target Node ID. If not defined, use the existing target ID	None
max_path_length	It is a traversal depth-limiting parameter used in graph engines to control how many levels (hops) deep a graph traversal algorithm will explore from a starting node.	100
output_table	Name of output table. If none, result will be returned from SP directly	None
temp_output_table	True: Output as volatile temporary table False: Output as perm table	True
show_query	Show the actual query	False

Example:

Python	<pre>graph_obj = td_graph_function.td_graph_object(database_name = 'graphdb', edge_table_name = 'graph_edge', edge_from_node_column_name = 'from_node', edge_to_node_column_name = 'to_node', edge_weight_column_name = 'edge_weight', node_table_name = 'graph_node', node_id_column_name = 'id', node_label_column_name = 'node_name') # Shortest Path graph_obj.td_graph_path_decode(edge_attributes = 'edge_weight', include_plot=True) all_path_from_1node</pre>									
Return	<table><tr><th></th><th>fullpath</th><th>weight</th></tr><tr><td>0</td><td>1,2,5</td><td>5.8</td></tr><tr><td>1</td><td>1,3,5</td><td>5.8</td></tr></table>		fullpath	weight	0	1,2,5	5.8	1	1,3,5	5.8
	fullpath	weight								
0	1,2,5	5.8								
1	1,3,5	5.8								

Function: Path Decode

Function: `td_graph_path_decode`

Description: Convert output path with node label

Parameter	Description	Default
input_table	Path table name, it's created by either <code>td_topology</code> or <code>td_shortest_path</code> call. If the tablename is not specified, the output table from last call will be used	None
input_table_kind	V: Volatile T: Perm Table	V
edge_attributes	Information included between nodes	None
output_table	Name of output table. If none, result will be returned from SP directly	None
temp_output_table	True: Output as volatile temporary table False: Output as perm table	True
include_plot	Include a Sankey diagram as output	False
show_query	Show the actual query	False

Example:

Python	<pre>all_path_from_1node = graph_obj.td_topology(source_id = 1, return_data = 'P', output_table = 'example_topology_path_vt', temp_output_table = True) path_label_df, fig = graph_obj.td_graph_path_decode(edge_attributes = 'edge_weight', include_plot=True) fig</pre>
Return	

Function: PageRank

Function: td_pagerank

Description: IT measures the importance or influence of each vertex (node) in a graph based on the structure of incoming edges

Parameter	Description	Default
directed	True: Directional network False: Bi-directional network	True
damping	Probability of following a link vs. jumping randomly.	0.85
max_iterations	Maximum number of iterations to run	100
tolerance	Stops when score change per iteration falls below this value	1e-8
show_query	Show the actual query	False

Example:

Python	<pre># Directed + Unweighted network graph_obj = td_graph_function.td_graph_object(edge_table_name = "graph_edges", edge_from_node_column_name = "source", edge_to_node_column_name = "target", edge_weight_column_name = None) df_du, iters_du = graph_obj.td_pagerank() print(df_du)</pre>																																	
Return	<table><thead><tr><th></th><th>node</th><th>pr_score</th></tr></thead><tbody><tr><td>23970</td><td>1</td><td>0.000156</td></tr><tr><td>21206</td><td>3</td><td>0.000103</td></tr><tr><td>45503</td><td>4</td><td>0.000089</td></tr><tr><td>48232</td><td>2</td><td>0.000087</td></tr><tr><td>11785</td><td>11</td><td>0.000086</td></tr><tr><td>17638</td><td>7</td><td>0.000083</td></tr><tr><td>8356</td><td>177</td><td>0.000082</td></tr><tr><td>38627</td><td>10</td><td>0.000073</td></tr><tr><td>17568</td><td>26</td><td>0.000068</td></tr><tr><td>19328</td><td>5</td><td>0.000066</td></tr></tbody></table>		node	pr_score	23970	1	0.000156	21206	3	0.000103	45503	4	0.000089	48232	2	0.000087	11785	11	0.000086	17638	7	0.000083	8356	177	0.000082	38627	10	0.000073	17568	26	0.000068	19328	5	0.000066
	node	pr_score																																
23970	1	0.000156																																
21206	3	0.000103																																
45503	4	0.000089																																
48232	2	0.000087																																
11785	11	0.000086																																
17638	7	0.000083																																
8356	177	0.000082																																
38627	10	0.000073																																
17568	26	0.000068																																
19328	5	0.000066																																

Conclusion

In-database graph functions represent a powerful paradigm shift in how organizations analyze complex, interconnected data. By executing graph algorithms directly within the Teradata engine — without moving data to an external graph system — teams gain the combined benefits of enterprise-scale data management and advanced network analysis in a single, unified platform.

Throughout this material, we explored the full spectrum of in-database graph capabilities: from fundamental traversal algorithms like **BFS and DFS**, to influence measurement via **PageRank**, to structural analysis through **path finding**, **dependency resolution**, and **pattern matching**. Each function is designed to operate at scale, processing millions of vertices and edges with the performance and reliability Teradata is known for.

The **teradataml** & **teradatage** packages bring these capabilities to Python data scientists through an intuitive, DataFrame-based API — making advanced graph analytics accessible without requiring deep expertise in graph theory or low-level SQL optimization.

Key Takeaways

01

PERFORMANCE

Algorithms run inside Teradata
— no data movement, no
external graph database
needed.

02

SCALABILITY

Algorithms run inside Teradata
— no data movement, no
external graph database
needed.

03

SIMPLICITY

Python-native API via teradataml
makes graph analytics
accessible to data scientists.

04

VERSATILITY

One engine covers traversal,
ranking, pathfinding, and
dependency analysis across all
domains.

About Teradata

At Teradata, we believe that people thrive when empowered with trusted information. That's why we built the most complete cloud analytics and data platform for AI.

By delivering harmonized data, Trusted AI, and faster innovation, we uplift and empower our customers—and our customers' customers—to make better, more confident decisions. The world's top companies across every major industry trust Teradata to improve business performance, enrich customer experiences, and fully integrate data across the enterprise. We drive positive impact for hundreds of millions of people every day around the world with faster, flexible data integration and trusted, cost-effective AI innovation.

